Test Writing & PYTHONPATH — Explained

pyspark-lakehouse-aws · Stage 1 (bronze ingestion) · 2026-08-06

The test-writing process, step by step

> **1. Start from what needs proving, not "let's write tests."**
>
> Two real bugs had already been found by hand: the zip code losing its leading zero, and review comments getting split across lines. Good tests exist to prove those bugs are fixed **and stay fixed** — so the plan started from "what breaks if this fix gets reverted," not from generic coverage.

> **2. Pick a test runner.**
>
> `pytest` is the standard Python testing tool — write plain functions starting with `test_`, and it finds and runs them automatically. Nothing fancier needed.

> **3. Set up shared setup ("fixtures") once, in `conftest.py`.**
>
> Every test needs a `SparkSession` to read a CSV with. Rather than creating one inside every test function, `conftest.py` defines it once:
>
> ```
> @pytest.fixture(scope="session")
> def spark():
>     session = SparkSession.builder.master("local[1]").appName("bronze-
> tests").getOrCreate()
>     yield session
>     session.stop()
> ```
>
> Any test function with a parameter named `spark` automatically receives this session — pytest wires it up by matching the name. `scope="session"` means it's built once and reused across every test in the run, since starting a Spark session is slow.

> **4. Decide what data to feed the tests.**
>
> Real files in `data/raw/` are the full dataset — big, and not written specifically to exercise the bug. Instead, each test builds a tiny 1–2 row CSV in memory, just big enough to reproduce the exact shape of the bug (a zip code starting with `0`; a comment field with a line break in it), and writes it to a temporary folder pytest creates and deletes automatically (`tmp_path` — another built-in fixture, wired up the same way as `spark`).

> **5. Every test follows Arrange–Act–Assert.**
>
> - **Arrange** — write the small CSV to a temp file.
> - **Act** — read it back using the same schema and options the real ingestion code uses.
> - **Assert** — check the result is what it should be.

> **6. Add one negative test — a regression guard.**
>
> `test_review_with_embedded_newline_corrupts_without_multiline_option` deliberately reruns the same input *without* the `multiLine` fix, and asserts it produces the wrong row count. If someone later removes the `multiLine` option during a cleanup, this test fails immediately instead of the bug silently coming back.

The PYTHONPATH issue, explained plainly

Think of `PYTHONPATH` as a list of folders Python checks whenever code says `import something`.

Spark code here runs two different ways:

- **spark-submit** (the real pipeline) — a smart launcher. Before your script even starts, it quietly adds Spark's own folders to `PYTHONPATH`, every time, no matter what.
- **Plain python3** (what `pytest` uses under the hood) — not smart. It only looks in whatever `PYTHONPATH` happens to be set to.

A few sessions back, to fix `ModuleNotFoundError: No module named 'config'`, `PYTHONPATH` was set to:

```
PYTHONPATH: /opt/app
```

This **replaced** the variable entirely — it didn't add `/opt/app` to whatever was already there, it became the *only* thing there. Invisible as a problem for weeks, because `spark-submit` doesn't care what `PYTHONPATH` was set to beforehand — it builds its own regardless.

`pytest` uses plain `python3`, though. When it tried `import pyspark`, it checked the one folder in `PYTHONPATH` (`/opt/app`) — no `pyspark` there — and failed.

Fix

List all three folders that need to be searchable, separated by colons:

```
PYTHONPATH: /opt/app:/opt/spark/python:/opt/spark/python/lib/py4j-0.10.9.7-src.zip
```

`/opt/app` is your own code (`config`, `src`). The other two are the exact folders `spark-submit` was quietly adding for you all along — now plain `python3` can find `pyspark` too, without needing `spark-submit` to hold its hand.

How to actually run the tests

One command from the Mac terminal (outside the container):

```
docker compose exec spark bash -lc "pip install -q pytest && cd /opt/app && python3 -m pytest tests/ -v"
```

Breaking that down:

- `docker compose exec spark bash -lc "..."` — run a command inside the already-running `spark` container.
- `pip install -q pytest` — make sure pytest is installed (see "the rough edge" below for why this is needed every time).
- `cd /opt/app && python3 -m pytest tests/ -v` — go to the project root, then run pytest against everything in the `tests/` folder. `-v` (verbose) prints each test's name and pass/fail individually instead of just a summary count.

If already inside the container (`docker compose exec spark bash`), just run the last two commands directly.

The "rough edge," explained

The `spark` container is built from Docker's official `apache/spark-py` image — meant for running Spark, so it doesn't come with `pytest` pre-installed.

Running `pip install pytest` inside the running container installs it into that container's *current, temporary* filesystem — not into the image itself. Containers are disposable: any time Docker has to recreate the container (which happens whenever `docker-compose.yml` changes — exactly what happened twice while wiring up the volume mount and `PYTHONPATH`), Docker throws away the old container and starts a fresh one from the original image. Fresh container = `pytest` is gone again, since it was never part of the image — only something bolted onto that one running instance.

> **Bottom line**
>
> Any time `docker compose up` reports `Container spark Recreated`, `pytest` needs reinstalling before tests will run. Not urgent to fix permanently — worth it only if a custom `Dockerfile` that bakes in `pytest` becomes worth the setup, i.e. if this starts happening often enough to be annoying.

The squiggly line in conftest.py

The Pylance warning `Cannot access attribute "master" for class "classproperty"` is a known false alarm with PySpark's type hints — `SparkSession.builder` is implemented in a way Pylance's static checker doesn't fully understand, so it misreports `.master(...)` as invalid even though it's the correct, documented API. Harmless — this exact code already ran and passed all three tests, so it's the editor being confused, not an actual bug.